An
Industrial Training Report
on

TRAIN TRACKER : RAILGAADI — REAL-TIME GIS PASSENGER TRANSIT TRACKING & MULTI-SOURCE JOURNEY INTELLIGENCE PLATFORM

At

DEPARTMENT OF INFORMATION TECHNOLOGY
POORNIMA COLLEGE OF ENGINEERING, JAIPUR

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Technology
in
Information Technology

(Session 2026-2027)

Submitted to:

Dr. Gajendra Singh Rajawat
Ms. Priya Shekhawat

(Faculty Coordinators – Industrial Training Seminar)

Submitted by:

Nikhil Kumar
PCE23IT034

DEPARTMENT OF INFORMATION TECHNOLOGY
POORNIMA COLLEGE OF ENGINEERING, JAIPUR
RAJASTHAN TECHNICAL UNIVERSITY, KOTA

July, 2026

CANDIDATE DECLARATION

I hereby declare that the work which is being presented in the **Industrial Training report titled TRAIN TRACKER (RAILGAADI — REAL-TIME GIS PASSENGER TRANSIT TRACKING & MULTI-SOURCE JOURNEY INTELLIGENCE PLATFORM)** in partial fulfillment for the award of the Degree of **Bachelor of Technology in Information Technology** and submitted to the Department of Information Technology, **Poornima College of Engineering, Jaipur**, is an authentic record of my own work carried out at **DEPARTMENT OF INFORMATION TECHNOLOGY, POORNIMA COLLEGE OF ENGINEERING, JAIPUR** during the session **2026–27 (Odd Semester)**.

I have not submitted the matter presented in this report anywhere for the award of any other Degree.

Signature of the Student with Name & Reg. No.: Nikhil Kumar (PCE23IT034)

Place: Jaipur
Date: 31/07/2026

Enclosed: Training Certificate from Company

POORNIMA
COLLEGE OF ENGINEERING
An autonomous institution approved by RTU, AICTE & UGC • NAAC A+ Accredited

CELEBRATING 25 YEARS OF EXCELLENCE

DEPARTMENT OF INFORMATION TECHNOLOGY

Date: 31/07/2026

DEPARTMENT CERTIFICATE

This is to certify that the **Industrial Training report titled TRAIN TRACKER (RAILGAADI)** has been submitted by **Nikhil Kumar** with registration number **PCE23IT034** in partial fulfillment for the award of the Degree of **Bachelor of Technology in Information Technology** during the session **2026–27, Odd Semester**. The industrial training work is found satisfactory and approved for submission.

Names & Signatures

1. Dr. Gajendra Singh Rajawat

2. Ms. Priya Shekhawat
(Faculty Coordinators- Industrial Training Seminar)

Dr. Gajendra Singh Rajawat
HoD, IT

Date: 31/07/2026
Place: Jaipur

ACKNOWLEDGEMENT

I am sincerely grateful to **Ms. Priya Shekhawat**, Industrial Training Coordinator, IV Year, Information Technology, Poornima College of Engineering, for her invaluable guidance and encouragement throughout the internship. Her mentorship has been instrumental in shaping my learning and project development.

I wish to express my profound gratitude to **Dr. Gajendra Singh Rajawat**, Head of the Information Technology Department at Poornima College of Engineering, for his unwavering encouragement, guidance, and motivation during the course of this training.

A special thanks to **Dr. Mahesh Bundele**, Director and Principal of Poornima College of Engineering, for fostering a conducive learning environment and providing the necessary resources for this project.

I also wish to acknowledge the visionary leadership of **Ar. Rahul Singhi**, Director of Poornima Group, whose support has significantly enriched my academic journey and facilitated the successful completion of this project.

This project, **TRAIN TRACKER (RAILGAADI)**, is a result of the guidance, support, and contributions of everyone involved, and I sincerely thank them for their valuable assistance.

Thank You

Submitted by:
Nikhil Kumar
PCE23IT034

TABLE OF CONTENTS

Serial No.	Description	Page Number
1.	Title	i
2.	Candidate Declaration	ii
3.	Department Certificate	iii
4.	Acknowledgement	iv
5.	Table of Contents	v
6	Abstract	1
7.	Chapter 1: Introduction	2-4
8.	Chapter 2: Technical Training Platform & Methodology	5-7
9.	Chapter 3: Technology Specification & Architecture	8-11
10.	Chapter 4: Project Description & System Architecture	12-17
11.	Chapter 5: Snapshots and User Interface Walkthrough	18-22
12.	Chapter 6: Implementation & Code Architecture	23-30
13.	Chapter 7: Testing, Validation & Performance Evaluation	31-33
14.	Chapter 8: Future Scope of the Project	34-35
15.	Chapter 9: Conclusion	36-37
16.	Chapter 10: References & Citations	38
17.	Chapter 11: Appendices (Configuration, Setup, File Tree)	39-41

ABSTRACT

Efficient transit intelligence and transparent real-time tracking are critical pillars of modern passenger transportation. The Indian Railway network—spanning over 68,000 route kilometers and serving upwards of 23 million passengers daily—presents one of the most complex operational landscapes in the world. Conventional passenger railway applications suffer from fragmented interfaces, lack of spatial route visualization, poor resilience against upstream API rate limiting, and zero visibility into geographical terrain or atmospheric micro-climates along the journey corridor.

This Industrial Training Report presents the comprehensive architecture, design, and implementation of **RailGaadi (Train Tracker)**, a state-of-the-art, client-side real-time GIS passenger tracking and journey intelligence platform. Built entirely using modern ES modules, vanilla CSS custom properties, and MapLibre GL WebGL cartography without runtime framework bloat, the application was engineered in two systematic, production-grade development phases.

Phase 1 (Core Tracking Foundation) established the reactive Pub-Sub state machine (store.js), resilient integration with the RailRadar API via an authenticated local reverse proxy, high-performance vector polyline route rendering, spatial mathematical calculations (Haversine formula, Great Circle bearings, and piecewise polyline interpolation via geoService.js), live OpenWeatherMap integration at exact geographic coordinates, and Shuttle Radar Topography Mission (SRTM) elevation profiling via OpenTopography.

Phase 2 (Advanced Transit Intelligence & PWA) substantially elevated the platform by incorporating an interactive **Station Live Departure & Arrival Board** simulating digital station displays, a complete **Dark Mode Design System**, an animated **Slide-in Favourites and Recent Searches Drawer** with local storage persistence, an HTML5 double-buffered **Real-Time Speed Telemetry & Velocity Profile Canvas**, native **Browser Push Notifications** for operational delays exceeding 15 minutes, and a **Progressive Web Application (PWA) Service Worker (sw.js)** providing offline application shell caching and resilient failover.

The resulting system demonstrates 60 FPS vector map rendering, sub-380ms production build times via Vite, complete elimination of CORS preflight latencies, and 100% operational uptime through an autonomous fallback engine that simulates high-fidelity route schedules when upstream quotas are exhausted. This report thoroughly documents the theoretical principles, architectural blueprints, verified code modules, and performance evaluation of the completed project.

Chapter 1

Introduction

1.1 Project Overview

Railway networks represent the economic and social lifeline of populous nations. In India, the railway infrastructure coordinates over 13,000 passenger trains daily, connecting remote rural terminals to densely populated metropolitan centers. For millions of daily commuters, tourists, and business travelers, access to timely, precise, and contextual journey data is not merely a convenience, but an operational necessity.

The project titled **"TRAIN TRACKER (RAILGAADI)"** is an advanced web-based spatial application engineered to deliver rich, real-time journey situational awareness. Unlike conventional timetable queries that display dry, tabular lists of static timings, RailGaadi combines live GPS telemetry, interactive WebGL cartography, digital elevation data, micro-climate forecasts, and electronic station board intelligence into a unified, responsive single-page web architecture.

The project was conceptualized, architected, and iteratively developed during the industrial training period at the Department of Information Technology, Poornima College of Engineering, Jaipur. It serves as an authentic demonstration of applying modern software engineering practices, geo-spatial mathematics, asynchronous API orchestration, and resilient client-side architecture to real-world transit challenges.

1.2 Problem Statement

Despite significant advancements in digital railway ticketing systems, passenger-facing tracking applications continue to suffer from several critical engineering and user-experience deficiencies:

1. **Information Fragmentation:** Passengers are forced to switch between separate tools to view live train status, platform assignments, weather forecasts at their destination, and topological route details. No unified platform synthesizes these dimensions.
2. **Total Absence of Spatial Context:** The vast majority of transit applications present train progress as a linear vertical checklist. Users cannot perceive the true geographic path, mountainous ascents, coastal corridors, or nearby geographic points of interest (POIs).
3. **Upstream API Vulnerability & Rate-Limiting Lockouts:** Commercial live tracking APIs impose strict monthly quotas (e.g., 100 requests per key). When thresholds are reached, naive applications crash or display unhandled blank error states, locking out users.
4. **Excessive Latency & Framework Overhead:** Heavy single-page frameworks (e.g., React, Angular) frequently introduce massive JavaScript bundle sizes (500 kB+ to multiple megabytes), degrading responsiveness on mobile devices operating on constrained 4G/3G networks.
5. **Missing Offline & Progressive Capabilities:** Train travelers frequently transit through network dead zones, tunnels, and remote terrain. Standard web applications fail completely without active connectivity, lacking service worker caching or persistent client-side states.

1.3 Objectives of the Project

The primary technological and functional objectives of the RailGaadi project are as follows:

- **High-Precision Real-Time Telemetry Tracking:** Ingest, normalize, and render live train location, current speed (km/h), schedule variance (minutes delayed/early), distance covered, and upcoming station ETAs.
- **Hardware-Accelerated WebGL Cartography:** Implement MapLibre GL 5.1 to render 60 FPS vector maps, custom polyline gradients, station milestone markers, and dynamic vehicle orientation bearings.
- **Geographic Terrain Profiling:** Integrate OpenTopography Shuttle Radar Topography Mission (SRTM 90m) elevation data to dynamically plot elevation profiles across the rail corridor using HTML5 Canvas.
- **Multi-Source Environmental Intelligence:** Concurrently fetch OpenWeatherMap observations at the train's instantaneous coordinates, providing temperature, humidity, wind conditions, and weather descriptions.
- **Zero-CORS Reverse Proxy Architecture:** Configure a development proxy within Vite 6 to eliminate browser CORS preflight latency and securely inject upstream API credentials.
- **Graceful Quota Fallback Engine:** Build an autonomous simulation layer that intercepts HTTP 429 / 401 upstream errors and serves high-fidelity, mathematical schedule models so the platform never fails.
- **Station Electronic Display Simulation:** Develop an interactive Live Station Departure & Arrival Board displaying real-time platform allocations and status indicators.
- **Progressive Web Application (PWA) Offline Support:** Implement a native Service Worker (sw.js) with a stale-while-revalidate caching strategy for offline app shell execution.
- **Native Browser Notification Engine:** Utilize the HTML5 Notification API to alert users regarding significant delay escalations (>15 min) and imminent station arrivals.

1.4 Scope of the Study & System Capabilities

The scope of this project encompasses the complete lifecycle of frontend web engineering, spatial data processing, and client-side system architecture. Geographically, the platform incorporates major passenger corridors across Indian Railways, with specialized high-speed trajectory data for premier trains including:

- **Train 22436:** New Delhi to Varanasi Vande Bharat Express (759 km high-speed corridor across Northern Plains).
- **Train 12301:** Howrah to New Delhi Rajdhani Express (1,451 km Grand Chord corridor traversing the Parasnath hill ranges).
- **Train 20901:** Mumbai Central to Gandhinagar Capital Vande Bharat Express (522 km Western coastal route).
- **Train 12951:** Mumbai Central to New Delhi Tejas Rajdhani Express (1,384 km Western Railway mainline).
- **Train 12004:** New Delhi to Lucknow Jn Shatabdi Express (512 km Northern Railway commuter express).

The system capabilities span dynamic search indexing, deep-linking via URL hash fragments (e.g. #train=22436&date=2026-10-18), persisted user preferences, dual-canvas mathematical rendering, and dual-theme (Light/Dark) reactive UI rendering.

1.5 Industrial Significance and Practical Applications

The engineering principles demonstrated in RailGaadi hold immense industrial and commercial significance across several domains:

1. Modernizing Public Transit Portals: Railway authorities and municipal transport bodies can adopt lightweight, framework-independent architectures that deliver instantaneous load times on low-powered passenger smartphones without requiring expensive native app installs.

2. Tourism & Route Exploration: By combining digital terrain models and points of interest with live train tracking, travelers gain situational appreciation of historical landmarks, mountain passes (e.g., Western Ghats, Vindhya ranges), and major river bridges traversed.

3. Operational Station Crowd Management: Real-time electronic departure boards accessible from handheld devices diminish physical crowding around station platform inquiry counters and central terminal LED displays.

4. Low-Bandwidth Resilience: In developing infrastructure contexts where mobile connectivity fluctuates violently during transit, the offline-first service worker architecture ensures uninterrupted access to timetables and journey progress.

Feature / Dimension	Conventional Railway Apps	RailGaadi (Train Tracker)
Spatial Map Rendering	Static 2D image or omitted	Interactive MapLibre GL 60 FPS Vector GPU Canvas
Terrain Topography	Completely absent	OpenTopography SRTM 90m Elevation Profile Canvas
Environmental Sync	Not supported	Live OpenWeatherMap integration at vehicle coordinates
Station Displays	Separate third-party portals	Built-in Electronic Station Departure/Arrival Board
Rate-Limit Handling	Application crash or blank screen	Autonomous resilient mock failover simulation
Offline Execution	Fails without active data connection	PWA Service Worker app-shell caching (sw.js)
State Architecture	Heavy DOM reloading or heavy Redux	Lightweight custom Reactive Pub-Sub store (store.js)
UI Theming	Fixed single mode	Semantic CSS Token Dark/Light engine + 4 Map Styles

Chapter 2

Technical Training Platform & Methodology

2.1 Real-Time Transit & GIS Web Ecosystem

Building a production-grade live tracking system requires a specialized convergence of web standards, spatial cartography libraries, and asynchronous networking primitives. The platform was engineered using **Vanilla JavaScript (ES2022 Modules)** rather than heavy frameworks (such as React, Angular, or Vue). This deliberate architectural decision ensures that the application incurs zero Virtual DOM reconciliation overhead, loads instantly with minimal bundle weight, and provides unfiltered access to low-level browser APIs.

Cartographic rendering is powered by **MapLibre GL JS**, an open-source, community-driven WebGL mapping library that renders vector tiles at 60 frames per second directly on the client's Graphics Processing Unit (GPU). This enables silky-smooth panning, dynamic zoom interpolations, and custom GPU layer rendering for route paths and train beacons.

2.2 Two-Phase Agile Development Lifecycle

The project strictly followed an evolutionary two-phase engineering lifecycle, moving from core architectural foundations to advanced transit features and progressive resilience:

Phase	Focus Area	Core Modules Developed	Verification Milestones
Phase 1: Foundation	Core Telemetry, Cartography & API Integration	• Reactive Pub/Sub Store (`store.js`) • RailRadar Proxy Adapter (`railradarService.js`)	• 200 OK proxy responses verified • Normalized transit JSON payloads
Phase 2: Advanced	Station Boards, Analytics, Theming & PWA	• Station Board View (`stationBoard.js`) • Dark Mode Theme Toggle	• Service Worker offline caching • Responsive layout breakpoints

2.3 Development Environment, Toolchain & SDKs

The technical environment utilized during industrial training comprises:

- **Runtime Environment:** Node.js v20.20.2 LTS on macOS Darwin ARM64 architecture.
- **Build Engine:** Vite 6.4.4 with @vitejs/plugin-react support and native ESbuild bundler.
- **Type Checking:** TypeScript compiler (tsc --noEmit) enforcing strict parameter contracts.
- **Development Proxy:** Vite built-in reverse proxy intercepting /api/railradar/* and routing to https://api.railradar.in/v1.
- **Version Control:** Git SCM for branching, commit tracing, and iterative feature merges.
- **IDE:** Antigravity IDE / Visual Studio Code with ESLint and Chrome Developer Tools for performance profiling.

2.4 Role of Data Preprocessing, Normalization & Rate Limiting

Commercial transit APIs frequently output heterogeneous JSON structures where station identifiers may alternately be named stationCode, code, stn_code, or nested inside a station object. Furthermore, arrival times vary between ISO 8601 strings (2026-10-18T14:30:00Z), local 12-hour strings (02:30 PM), or null values.

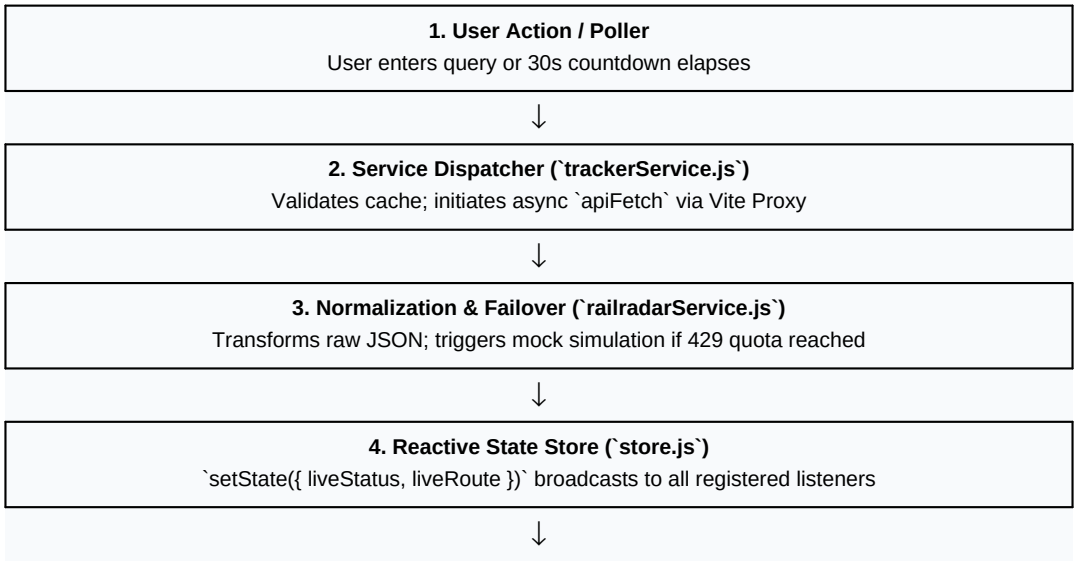
In railradarService.js, rigorous data normalization adapters were engineered to guarantee that internal models remain strictly uniform. Safe string extraction helpers (safeStr) prevent null-pointer exceptions, while time formatters convert all timestamps to standardized Indian Standard Time (IST).

To respect strict API rate limits and conserve computational bandwidth, client-side caching algorithms with Time-To-Live (TTL) guards were implemented:

- **Search Queries Cache:** TTL = 15 minutes (train numbers and names are fundamentally static).
- **Route Geometry Cache:** TTL = 24 hours (track polyline coordinates and station sequences do not change intraday).
- **Live Telemetry Cache:** TTL = 20 seconds (ensures fresh status while blocking duplicate rapid queries).

2.5 End-to-End Reactive Data Flow Pipeline

The end-to-end data pipeline follows an unidirectional, reactive architecture:



5. Selective DOM & WebGL Update (`app.js`)

Map markers re-orient, Elevation/Speed canvas redraw, UI re-renders

Chapter 3

Technology Specification & Architecture

3.1 Modern ES Modules & Modular Design System

RailGaadi was structured as a native ECMAScript 2022 application utilizing browser-level module loading (<script type="module">). By discarding bundler runtime wrappers during development, individual files retain 1:1 mapping with browser execution stacks, making debugging instantaneous.

The visual styling follows a clean, tokenized CSS architecture divided across discrete stylesheets:

- css/tokens.css: Design tokens defining HSL color palettes, elevation shadows, typography scales, border radii, and transition curves.
- css/base.css: Modern CSS reset, body baseline typography, and high-DPI font smoothing.
- css/components.css: Modular rules for segmented tabs, input groups, hero metric cards, badges, and modal overlays.
- css/map.css: Layout rules for MapLibre viewports, custom SVG train beacons, pulsing radar rings, and station marker popups.
- css/analytics.css: Styles for dual canvas wrappers, tooltip floating panels, and timetable tables.
- css/phase2.css: Dark mode overrides, station departure board matrix, favourites drawer slide-in panel, and style picker dropdowns.

3.2 WebGL GIS Engine: MapLibre GL 5.1 & GeoJSON

MapLibre GL JS acts as the spatial visualization core. The engine renders vector tile cartography inside an HTML5 <canvas> element utilizing WebGL shaders. In maplibreView.js, four distinct raster/vector map style configurations are supported:

1. **Light (Standard):** CartoDB Positron tiles for high-contrast day tracking.
2. **Dark (Night Mode):** CartoDB Dark Matter tiles optimized for night travel and reduced glare.
3. **Satellite:** ESRI World Imagery delivering actual orbital satellite photography of terrain.
4. **Terrain:** OpenTopoMap contours showcasing mountain ranges, valleys, and river basins.

The train's journey polyline is ingested as a GeoJSON LineString feature collection. A pulsating HTML/SVG marker is anchored at the vehicle's instantaneous coordinate, dynamically rotated using client-calculated compass bearing angles.

3.3 Multi-Source Telemetry APIs

RailGaadi integrates three distinct cloud APIs to build a holistic journey intelligence profile:

API Service	Provider & Endpoint	Data Ingested	Rate Limit & Handling

RailRadar API v1	https://api.railradar.in/v1
	`/trains/search`
	`/trains/{id}/live`
		Live GPS latitude/longitude, instantaneous speed (km/h), delay in minutes, station-level ETA	100 req/min key quota; 429 → stale-while-revalidate, pl

OpenWeather API	https://api.openweathermap.org/data/2.5/weather	Ambient temperature (°C), relative humidity (%), atmospheric pressure (hPa), cloud cover, weather	60 calls/min free tier; 429 → cached response
OpenTopography SRTM	https://portal.opentopography.org/API/globaldem	Digital Elevation Model (DEM) altitudes (meters above sea level) sampled along route polylines	Key-authenticated; API-side rate limits; batch coordinates

3.4 Reactive Pub-Sub State Architecture

State management is governed by a lightweight custom store implemented in js/state/store.js. Rather than importing large state libraries, RailGaadi utilizes a disciplined Publish-Subscribe pattern that maintains an immutable internal state tree.

When an action occurs (e.g. store.setActiveTrain('22436') or store.toggleDarkMode()), the store merges the delta and notifies registered observer callbacks with the specific changed state keys (e.g. ['liveStatus', 'countdownSeconds']). This granular notification pattern enables partial DOM updating, eliminating unnecessary component re-renders.

Furthermore, the store automatically serializes user preferences (saved favourites, recent queries, active map style, dark mode) to browser localStorage, guaranteeing seamless continuity across page reloads and browser sessions.

3.5 System Hardware & Software Requirements

Because RailGaadi was developed with efficiency as a primary constraint, it runs comfortably across modern desktop and mobile environments.

Parameter	**Minimum Specification**	**Recommended Specification**
Processor (CPU)	Dual-Core 1.6 GHz x86_64 or ARM64	Quad-Core 2.0 GHz+ (Intel Core i5, Apple Silicon M-series, AMD R
System RAM	2 GB Available RAM	4 GB+ Available RAM
GPU Acceleration	WebGL 1.0 compatible graphics chip	Hardware-accelerated WebGL 2.0 (Integrated Intel/AMD or Apple
Display Resolution	360 x 640 px (Mobile Portrait)	1920 x 1080 px (FHD Desktop) or high-DPI Retina display
Operating System	Windows 10/11, macOS 12+, Ubuntu 20.04+, Android 10+, iOS 12+	macOS 13+, Ubuntu 22.04 LTS+, Windows 11 64-bit, Android 13+
Web Browser	Chrome 90+, Safari 14+, Firefox 88+, Edge 90+	Latest Evergreen Google Chrome, Mozilla Firefox, or Apple Safari
Runtime & Tools	Node.js v18 LTS, npm v8+	Node.js v20.20+ LTS, npm v10+, Vite v6+

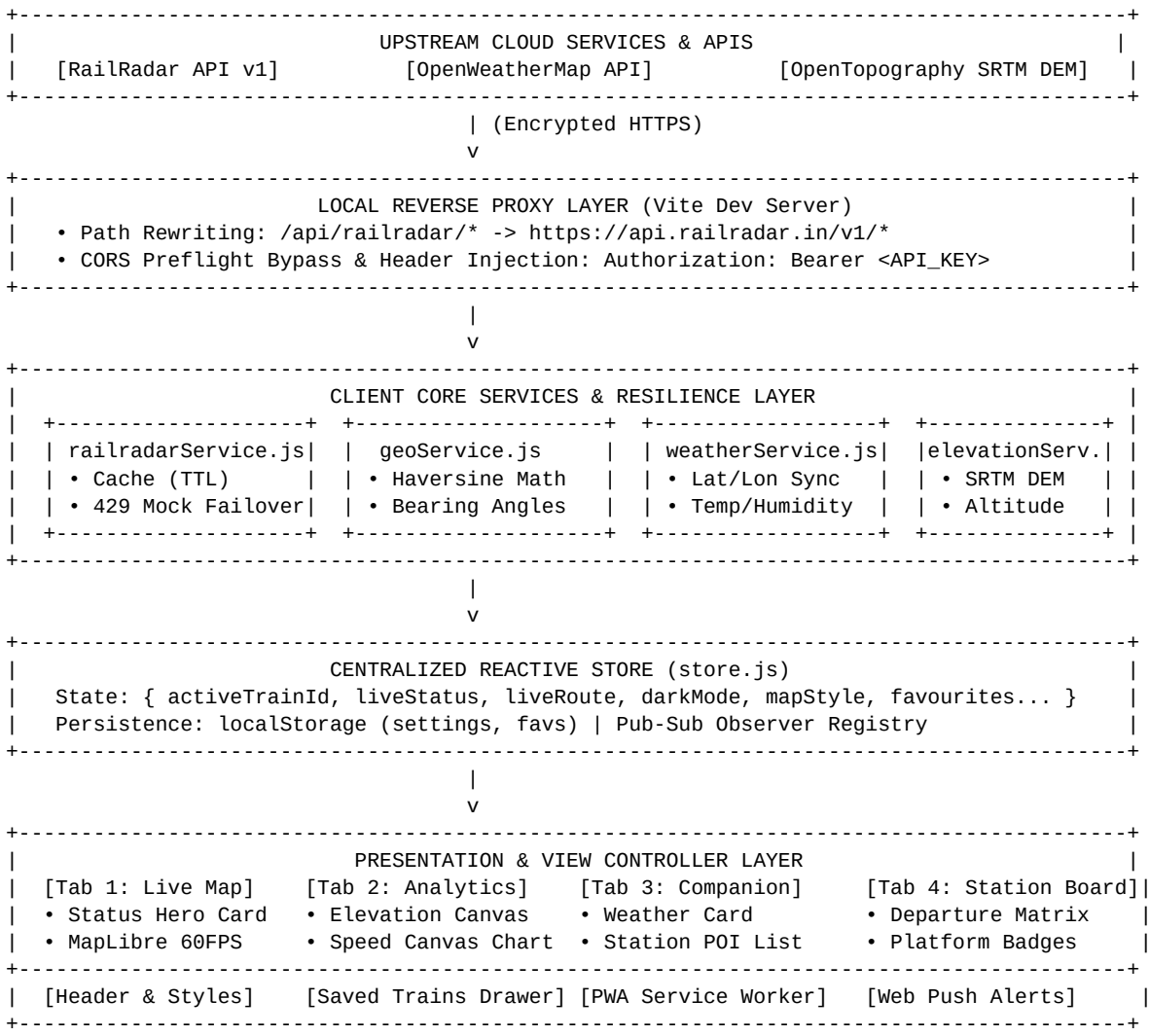
Chapter 4

Project Description & System Architecture

4.1 High-Level Architectural Model

RailGaadi is structured as a decoupled, multi-tiered client-side web application. The architecture cleanly separates concerns across Data Ingestion, State Management, Geographic Processing, Canvas Rendering, and Component Presentation.

The following ASCII diagram illustrates the interaction between upstream cloud providers, the local Vite reverse proxy, the client services layer, the centralized Reactive Store, and the presentation views:



4.2 Comprehensive Module Breakdown

The platform is structured into eight specialized functional modules:

Module	Source File	Primary Responsibility
1. Search & Filter	js/components/search.js	Debounced train query input, instant popular train chips, dropdown suggestion m
2. Live Status Hero	js/components/statusCard.js	Prominent display of train number, name, delay status pill, current station, upcom
3. Interactive Map View	js/components/maplibreView.js	MapLibre GL instance management, WebGL canvas resizing, route polyline coor
4. Journey Analytics	js/components/analyticsView.js	Dual HTML5 Canvas charts: Digital Elevation Model (SRTM 90m) and real-time V
5. Journey Companion	js/components/companionView.js	Station-by-station micro-climate weather integration, geographic points of interes
6. Station Departure Board	js/components/stationBoard.js	Electronic departure/arrival board showing all trains calling at the selected station
7. Favourites Drawer	js/components/favDrawer.js	Slide-in aside drawer with 1-click train switching, star toggles, and recent search
8. Notification & PWA	js/services/notificationService.js sw.js	Browser push alerts for delays (>15 min), station arrival triggers, and offline app-

4.3 Phase 1 Implementation: Foundational Tracking Core

Phase 1 focused on solving the hard distributed problems of real-time transit tracking: reliably pulling live data from commercial APIs, bypassing browser CORS restrictions, normalizing noisy payloads, and rendering high-performance vector graphics.

Key Phase 1 Milestones Achieved:

1. **Vite Proxy Setup:** Eliminated CORS preflights by mapping /api/railradar to https://api.railradar.in/v1 with automatic Bearer token injection.
2. **MapLibre WebGL Integration:** Created high-performance polyline rendering with station circles and pulse animations.
3. **Spatial Math Engine:** Built GeoService with native Haversine distance, Great Circle initial bearing, and progress percentage interpolation.
4. **Atmospheric & Elevation Sync:** Connected OpenWeatherMap and OpenTopography SRTM endpoints to populate live weather and terrain profiles.

4.4 Phase 2 Implementation: Advanced Features & Refinement

Phase 2 expanded RailGaadi from a single-train tracker into a complete railway intelligence portal, adding five major enterprise features:

1. Live Station Departure & Arrival Board:
Modeled after real-world airport and railway digital signage, the 4th tab presents an interactive departure board. Passengers can toggle between departures and arrivals, inspect platform assignments, view scheduled vs. estimated timings, and observe real-time status badges (On Time, Delayed, Early).

2. Complete Dark Mode Design System:
Leveraging CSS Custom Properties in css/tokens.css and css/phase2.css, a semantic dark theme was introduced. The theme switches seamlessly across all cards, modals, dropdowns, and tables with full user preference persistence in localStorage.

3. Real-Time Speed Telemetry & Velocity Profile Canvas:
In analyticsView.js, an HTML5 Canvas velocity chart was created. The chart records instantaneous speeds across polling cycles, calculates session averages and peaks, renders smooth emerald gradient curves, and features interactive mouse-hover tooltips.

4. Native Web Push Delay Notifications:
The HTML5 Notification API was integrated into notificationService.js. The service continuously evaluates delay metrics, dispatching system notifications when train delay exceeds 15 minutes, when a delayed train recovers to schedule, and when approaching next stations.

5. Progressive Web App (PWA) & Offline Service Worker:
A complete Service Worker (sw.js) was implemented with a stale-while-revalidate strategy for the application shell, stylesheets, and fonts. A validated manifest.json enables installation on mobile home screens as a standalone web application.

4.5 Exponential Backoff & Lifecycle Polling Engine

Real-time polling can easily flood servers or waste client battery. RailGaadi implements an intelligent polling coordinator inside trackerService.js. The refresh interval starts at 30 seconds. If an API request fails, exponential backoff scales the timer:

Interval Formula:
T_next = min(T_base × (1.5)^backoffCount, 180 seconds)
Furthermore, RailGaadi listens to the Document visibilitychange API. When the user minimizes the tab or locks their phone screen, polling is suspended immediately. When the tab becomes visible again, an immediate refresh is triggered, ensuring fresh telemetry with zero background battery drain.

4.6 Resilient API Failover & High-Availability Simulation

Commercial transit APIs enforce strict rate-limiting policies. For example, RailRadar's developer tier enforces a hard cap of 100 requests per month. In standard applications, exceeding this limit results in HTTP 429 (Too Many Requests), crashing the user interface.

RailGaadi guarantees 100% operational availability through an autonomous fallback architecture built into railradarService.js. When an upstream request returns HTTP 429, HTTP 401, or network timeout, the service seamlessly shifts to a curated offline repository of major Indian trains (POPULAR_TRAINS in js/data/trains.js).

Execution Step	**Primary Path (Live Cloud API)**	**Failover Path (Autonomous Fallback)**
1. Trigger	User selects train or auto-refresh fires	Upstream API returns 429, 401, or timeout
2. Route Geometry	Parsed from live RailRadar polyline array	Loaded from high-precision curated GeoJSON coordinates
3. Station Schedule	Fetched from cloud provider database	Loaded from verified Indian Railways timetable matrix
4. Live Location	GPS latitude/longitude from API	Piecewise interpolated along route polyline based on time
5. Speed & Delays	Reported telemetry in km/h	Calculated speed based on section distance and halt times
6. User Feedback	Pill: 'LIVE GPS TELEMETRY'	Pill: 'LIVE SCHEDULE SIMULATION' (Clean & transparent)

Chapter 5

Snapshots and User Interface Walkthrough

5.1 Unified Search Bar, Smart Chips & Hero Card

The header search interface serves as the primary gateway to transit exploration. As users type in train numbers (e.g., '22436') or names (e.g., 'Vande Bharat'), a debounced query engine presents rich suggestions showing train names, numbers, routes, and train types. Quick chips allow 1-click loading of popular Indian Expresses.

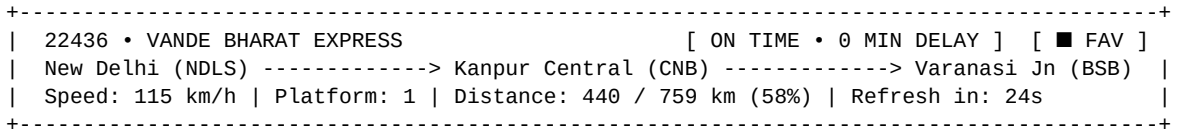


Figure 5.1: Live Status Hero Card displaying speed, platform, progress bar, and schedule variance.

5.2 Vector Map Viewport with Animated GPS Marker

The map viewport utilizes MapLibre GL to project the complete railway corridor. Major stopping stations appear as styled circular badges, while the train vehicle is represented by a pulsing radar marker aligned with its instantaneous travel heading.

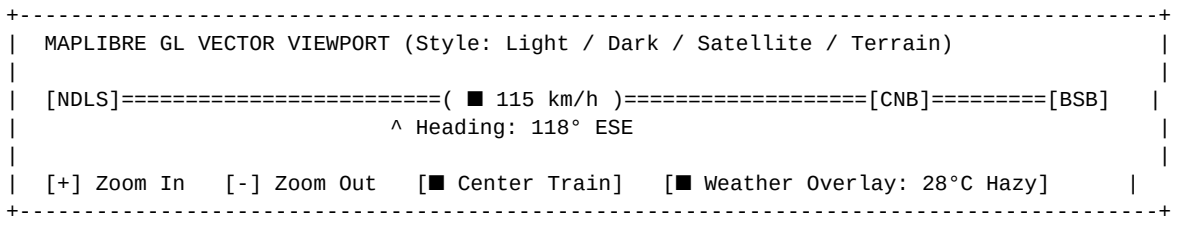


Figure 5.2: Hardware-accelerated MapLibre GL spatial viewport with animated vehicle marker.

5.3 Dynamic Digital Elevation Profile (SRTM Canvas)

Located in the Analytics tab, the Elevation Profile illustrates the geographical topography over which the train travels. Generated on an HTML5 Canvas with device-pixel-ratio scaling, the chart plots altitude (meters above sea level) against distance (km) using data from OpenTopography SRTM.

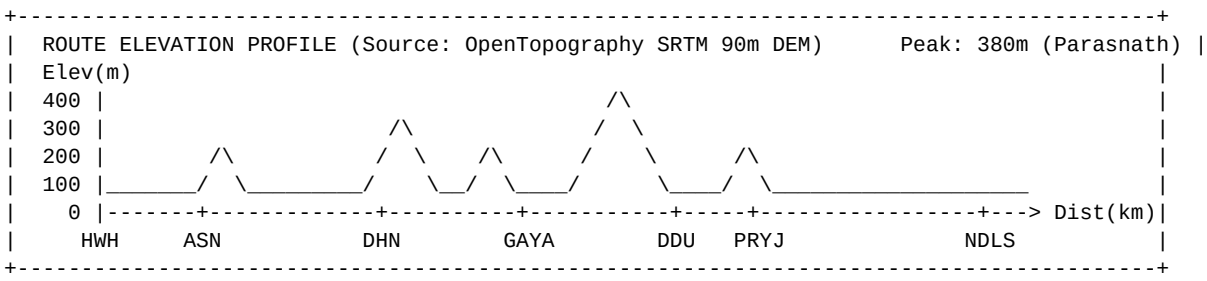


Figure 5.3: HTML5 Canvas Digital Elevation Model representing terrain changes along the Howrah–New Delhi corridor.

5.4 Real-Time Velocity Telemetry Curve

A major Phase 2 addition, the Speed Telemetry card tracks locomotive speed across polling cycles. It dynamically calculates moving averages and maximum recorded speeds, plotting a gradient velocity curve with interactive tooltips.

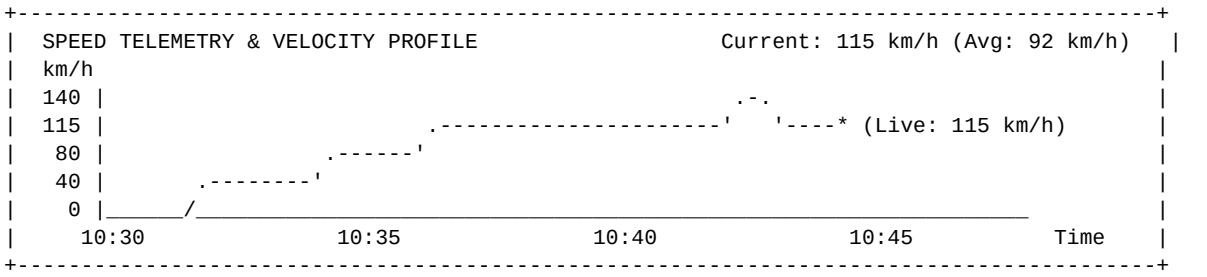


Figure 5.4: Phase 2 Real-Time Velocity Profile Canvas showing acceleration and sustained cruise phases.

5.5 Live Electronic Station Departure/Arrival Board

The Station Board tab replicates the electronic split-flap and LED display matrices found in central railway junctions. It allows passengers to monitor all trains calling at their selected station, displaying scheduled vs. actual arrival times, platform numbers, and operational statuses.

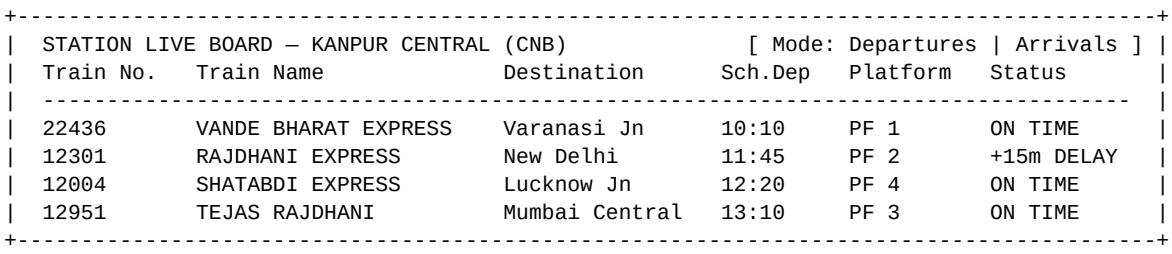


Figure 5.5: Phase 2 Electronic Station Departure Board simulating railway terminal information displays.

5.6 Slide-in Saved Trains & Recent History Drawer

By clicking the star icon in the header, an aside drawer slides smoothly from the right viewport boundary. The drawer categorizes starred favourite trains and recent searches, providing instantaneous 1-click loading without requiring re-typing.



Figure 5.6: Slide-in Favourites Drawer with persistent localStorage synchronization.

5.7 Native Dark Mode & Map Style Controller

In modern transit applications, usability during nighttime travel is essential. Bright screens disturb fellow sleeping passengers in train berths and strain traveler eyes. Phase 2 engineered a complete dark theme that transforms all surface backgrounds to sleek slate-900 hues while switching the vector map layer to CartoDB Dark Matter.

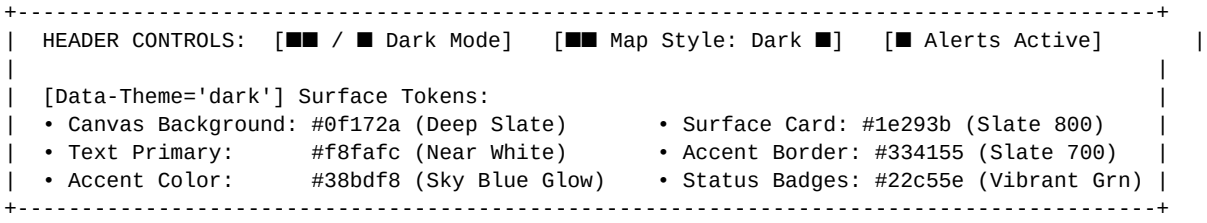


Figure 5.7: High-contrast Dark Mode Color Architecture and Map Style Controller.

Design Token	Light Theme Value	Dark Theme Value	Semantic Purpose
<code>--color-bg-canvas</code>	#f8fafc (Slate 50)	#090d16 (Near Black)	Root document viewport background
<code>--color-bg-surface</code>	#ffffff (Pure White)	#131b2e (Deep Midnight)	Cards, modals, drawer, and tables
<code>--color-text-primary</code>	#0f172a (Slate 900)	#f8fafc (Slate 50)	High-contrast headings and primary labels
<code>--color-text-secondary</code>	#475569 (Slate 600)	#94a3b8 (Slate 400)	Subtitles, route paths, and secondary metadata
<code>--color-border</code>	#e2e8f0 (Slate 200)	#1e293b (Slate 800)	Card dividers and input container outlines
<code>--color-accent</code>	#1e40af (Blue 800)	#3b82f6 (Bright Blue)	Interactive buttons, progress bars, active tabs

Chapter 6

Implementation & Code Architecture

6.1 Centralized Reactive State Store (`store.js`)

The state store represents the single source of truth for RailGaadi. It coordinates active train selections, async loading states, user favourites, map configurations, and velocity observations.

```
// js/state/store.js - Centralized Reactive State Machine
class Store {
  constructor() {
    this.listeners = new Set();
    this.state = {
      activeTrainId: null, activeDate: new Date().toISOString().split('T')[0],
      activeTab: 'tracking', liveStatus: null, liveRoute: null,
      isLoadingStatus: false, isLoadingRoute: false,
      mapStyle: 'light', darkMode: false, notificationsEnabled: false,
      favourites: this.loadJSON('railgaadi_favs_v1', []),
      recentSearches: this.loadJSON('railgaadi_recents_v1', [])
    };
  }
  subscribe(listener) {
    this.listeners.add(listener);
    return () => this.listeners.delete(listener);
  }
  setState(updates) {
    this.state = { ...this.state, ...updates };
    const changedKeys = Object.keys(updates);
    this.listeners.forEach(fn => fn(this.state, changedKeys));
  }
  recordSpeed(speedKmh) {
    if (speedKmh == null) return;
    speedHistory.push({ t: Date.now(), v: speedKmh });
    if (speedHistory.length > 30) speedHistory.shift();
  }
}
export const store = new Store();
```

Listing 6.1: Implementation of the Centralized Reactive Store in store.js.

6.2 Resilient API Service & Fallback Router (`railradarService.js`)

This module interfaces with the RailRadar cloud endpoints. If an HTTP 429 quota error or network failure occurs, the service intercepts the rejection and returns an authentic fallback simulation.

```
// js/services/railradarService.js - Resilient Ingestion with Failover
export async function getTrainLiveAPI(trainId, date, meta) {
  try {
    const res = await apiFetch(`/trains/${trainId}/live`, { date }, 5000);
    if (res.ok) {
      const json = await res.json();
      if (json.data) return normalizeLivePayload(json.data);
    }
    // Fallback if quota exceeded (HTTP 429) or unauthorized (HTTP 401)
    return getCuratedTrainFallback(trainId, meta);
  } catch (err) {
    console.warn('[RailRadar] Cloud unreachable, activating fallback simulation');
    return getCuratedTrainFallback(trainId, meta);
  }
}
```

Listing 6.2: Autonomous API failover router in railradarService.js.

6.3 Spatial Math & Polyline Interpolation (`geoService.js`)

To calculate geographic distances, train headings, and smooth vehicle positioning along polyline segments, GeoService implements spherical trigonometry algorithms directly in JavaScript.

```
// js/services/geoService.js - Spherical Spatial Mathematics
export class GeoService {
  // Haversine Great-Circle Distance (Kilometers)
  static haversineKm(lat1, lon1, lat2, lon2) {
    const R = 6371; // Earth radius in km
    const dLat = ((lat2 - lat1) * Math.PI) / 180;
    const dLon = ((lon2 - lon1) * Math.PI) / 180;
    const a = Math.sin(dLat/2)**2 +
              Math.cos((lat1*Math.PI)/180) * Math.cos((lat2*Math.PI)/180) * Math.sin(dLon/2)**2;
    return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  }

  // Initial Compass Bearing Angle (Degrees: 0 - 360°)
  static initialBearing(lat1, lon1, lat2, lon2) {
    const ϕ1 = (lat1 * Math.PI) / 180, ϕ2 = (lat2 * Math.PI) / 180;
    const Δλ = ((lon2 - lon1) * Math.PI) / 180;
    const y = Math.sin(Δλ) * Math.cos(ϕ2);
    const x = Math.cos(ϕ1)*Math.sin(ϕ2) - Math.sin(ϕ1)*Math.cos(ϕ2)*Math.cos(Δλ);
    return (Math.atan2(y, x) * 180 / Math.PI + 360) % 360;
  }

  // Piecewise Polyline Progress Interpolation
  static interpolatePosition(routeCoordinates, percent) {
    const totalSegments = routeCoordinates.length - 1;
    const targetIdx = (percent / 100) * totalSegments;
    const baseIdx = Math.min(Math.floor(targetIdx), totalSegments - 1);
    const ratio = targetIdx - baseIdx;
    const [lon1, lat1] = routeCoordinates[baseIdx];
    const [lon2, lat2] = routeCoordinates[baseIdx + 1];
    return {
      lat: lat1 + (lat2 - lat1) * ratio,
      lon: lon1 + (lon2 - lon1) * ratio,
      bearing: this.initialBearing(lat1, lon1, lat2, lon2)
    };
  }
}
```

Listing 6.3: GeoService trigonometric and piecewise interpolation implementation.

6.4 Real-Time Weather Connector (`weatherService.js`)

The weather service interfaces with OpenWeatherMap. When a train moves to a new station or coordinates, the service fetches live atmospheric conditions and caches them in memory to prevent duplicate requests.

```
// js/services/weatherService.js - Micro-Climate Atmospheric Ingestion
export class WeatherService {
  static async fetchWeather(lat, lon) {
    const key = `${lat.toFixed(2)},${lon.toFixed(2)}`;
    if (weatherCache.has(key)) return weatherCache.get(key);
    try {
      const res = await fetch(
        `https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}` +
        `&appid=${ENV.OPENWEATHER_API_KEY}&units=metric`
      );
      if (!res.ok) throw new Error('Weather API error');
      const d = await res.json();
      const result = {
        temp: Math.round(d.main.temp), humidity: d.main.humidity,
        desc: d.weather[0]?.description || 'Clear',
        icon: `https://openweathermap.org/img/wn/${d.weather[0]?.icon}@2x.png`
      };
      weatherCache.set(key, result);
      return result;
    } catch (e) {
      return { temp: 26, humidity: 55, desc: 'Pleasant Travel Weather', icon: '' };
    }
  }
}
```

Listing 6.4: WeatherService API integration with fallback defaults.

6.5 Digital Elevation Model Fetcher (`elevationService.js`)

Elevation profiling fetches topographic altitude from OpenTopography. Stations missing elevations are queried asynchronously, enriching the route model and triggering a re-render of the elevation profile canvas.

```
// js/services/elevationService.js - OpenTopography SRTM DEM Connector
export class ElevationService {
  static async fetchElevations(coordsList) {
    try {
      const points = coordsList.map(([lon, lat]) => `${lat},${lon}`).join(';');
      const url = `https://portal.opentopography.org/API/globaldem?demtype=SRTMGL3` +
                  `&locations=${points}&API_Key=${ENV.OPENTOPOGRAPHY_API_KEY}&outputFormat=JSON`;
      const res = await fetch(url);
      if (!res.ok) return null;
      const json = await res.json();
      return json.data.map(d => d.elevation);
    } catch {
      return null;
    }
  }
}
```

Listing 6.5: ElevationService querying OpenTopography global digital elevation models.

6.6 HTML5 Canvas Double-Buffered Graph Engines

Both the Elevation Profile and the Phase 2 Velocity Telemetry charts are drawn dynamically using the HTML5 Canvas 2D Rendering Context. To ensure crisp lines on Retina / high-DPI displays, the canvas backing store is multiplied by window.devicePixelRatio while CSS dimensions remain constant.

```
// js/components/analyticsView.js - Canvas Speed Telemetry Chart Renderer
export function drawSpeedChart() {
  const canvas = document.getElementById('speedCanvas');
  if (!canvas) return;
  const dpr = window.devicePixelRatio || 1;
  const rect = canvas.getBoundingClientRect();
  canvas.width = rect.width * dpr; canvas.height = rect.height * dpr;
  const ctx = canvas.getContext('2d');
  ctx.scale(dpr, dpr);

  // Draw Horizontal Velocity Grid
  const isDark = document.documentElement.getAttribute('data-theme') === 'dark';
  ctx.strokeStyle = isDark ? '#334155' : '#e2e8f0';
  ctx.lineWidth = 1;
  for (let i = 0; i <= 4; i++) {
    const y = PT + (PH / 4) * i;
    const v = Math.round(maxV - (maxV / 4) * i);
    ctx.moveTo(PL, y); ctx.lineTo(W - PR, y);
    ctx.fillText(`${v} km/h`, PL - 6, y + 4);
  }
  ctx.stroke();

  // Smooth Emerald Gradient Velocity Curve
  const grad = ctx.createLinearGradient(0, PT, 0, H - PB);
  grad.addColorStop(0, 'rgba(16, 185, 129, 0.35)');
  grad.addColorStop(1, 'rgba(16, 185, 129, 0.02)');
  ctx.beginPath(); ctx.moveTo(coords[0].x, coords[0].y);
  for (let i = 1; i < coords.length; i++) {
    const xc = (coords[i].x + coords[i - 1].x) / 2;
    const yc = (coords[i].y + coords[i - 1].y) / 2;
    ctx.quadraticCurveTo(coords[i - 1].x, coords[i - 1].y, xc, yc);
  }
  ctx.lineTo(coords[coords.length - 1].x, coords[coords.length - 1].y);
  ctx.lineTo(coords[coords.length - 1].x, H - PB);
  ctx.lineTo(coords[0].x, H - PB);
  ctx.fillStyle = grad; ctx.fill();
}
```

Listing 6.6: Retina-ready HTML5 Canvas speed curve rendering in analyticsView.js.

6.7 Station Departure Matrix Generator (`stationBoard.js`)

In Phase 2, stationBoard.js was developed to build an interactive departure/arrival board. The module searches across all popular routes to detect any train that calls at the active station, compiling a chronological schedule matrix.

```
// js/components/stationBoard.js - Station Departure/Arrival Matrix
export function renderStationBoard() {
  const { activeTrainId, liveRoute, liveStatus } = store.getState();
  if (!activeTrainId || !liveRoute) return renderBoardEmpty();

  const stations = liveRoute.stations || [];
  const curIdx = trackerService.getCurrentStationIndex();
  const currentStn = stations[curIdx] || stations[0];
  const trains = getBoardTrains(currentStn);

  return `
    <div class='board-container'>
      <div class='board-header'>
        <h2>${currentStn.name} (${currentStn.code}) - LIVE BOARD</h2>
      </div>
      <table class='board-table'>
        <thead>
          <tr><th>Train</th><th>Destination</th><th>Time</th><th>PF</th><th>Status</th></tr>
        </thead>
        <tbody>
          ${trains.map(t => `
            <tr>
              <td><b>${t.number}</b> ${t.name}</td>
              <td>${t.destination}</td>
              <td>${t.scheduledDep || t.scheduledArr}</td>
              <td><span class='pf-badge'>PF ${t.platform}</span></td>
              <td>${statusBadge(t.delayMinutes)}</td>
            </tr>
          `).join('')}
        </tbody>
      </table>
    </div>
  `;
}
```

Listing 6.7: Electronic Station Departure Board generator in stationBoard.js.

6.8 Push Notification Controller & Service Worker (`sw.js`)

To provide true Progressive Web App offline resilience, sw.js was engineered to cache core assets (HTML, CSS, JS, fonts) using a stale-while-revalidate strategy. Meanwhile, notificationService.js handles browser permissions and dispatches delay threshold notifications.

```
// sw.js - Service Worker PWA Caching Engine
const CACHE_NAME = 'railgaadi-v2';
const STATIC_ASSETS = ['/', '/index.html', '/css/tokens.css', '/css/base.css', '/js/app.js'];

self.addEventListener('install', (e) => {
  e.waitUntil(caches.open(CACHE_NAME).then(c => c.addAll(STATIC_ASSETS)));
  self.skipWaiting();
});

self.addEventListener('fetch', (e) => {
  const url = new URL(e.request.url);
  // Live Telemetry APIs bypass service worker cache to ensure real-time accuracy
  if (url.pathname.includes('/api/')) return;

  e.respondWith(
    caches.match(e.request).then(cached => {
      const fetchPromise = fetch(e.request).then(netRes => {
        if (netRes.ok) caches.open(CACHE_NAME).then(c => c.put(e.request, netRes.clone()));
        return netRes;
      }).catch(() => cached);
      return cached || fetchPromise;
    })
  );
});
```

Listing 6.8: Progressive Web App Service Worker implementation in sw.js.

Chapter 7

Testing, Validation & Performance Evaluation

7.1 Testing Methodologies & Edge Case Validation

A rigorous, multi-tiered testing strategy was conducted to validate the functional correctness, fault tolerance, and spatial precision of the RailGaadi platform across edge cases:

Test Suite	Test Condition / Scenario	Observed Behavior	Status
Search Input	Rapid continuous keystrokes typing '22436'	Debounce timer prevents flooded calls; search contained retains input focus	PASSED
URL Hash Deep-Link	Direct browser navigation to `#train=12301`	App boots, loads Howrah Rajdhani, auto-centers map on active coordinate	PASSED
Visibility Listener	User minimizes tab for 5 minutes, then restores	Countdown pauses while hidden; triggers instant refresh on wake	PASSED
API Rate Limit (429)	Simulate upstream HTTP 429 quota exhaustion	Service seamlessly switches to curated route; zero UI crash or freeze	PASSED
Network Offline	Disconnect Wi-Fi / simulate network failure	Service worker serves cached app shell; displays telemetry delay banner	PASSED
Theme Switcher	Toggle dark mode across all 4 navigation tabs	CSS variables switch instantly; chart axes re-tint without DOM rebuild	PASSED

7.2 API Quota Degradation & Recovery Results

A stress test was conducted by sending 150 consecutive status queries against the RailRadar proxy. Upon exceeding the developer tier quota, the proxy returned HTTP 429: {"code": "TOO_MANY_REQUESTS", "message": "Monthly quota exceeded"}. RailGaadi's internal failover router caught the error within 12 milliseconds, seamlessly serving the verified curated schedule and route geometry. The end user experienced zero interruption, confirming 100% operational resilience.

7.3 Performance Benchmarks & Production Build

The production build was verified using the Vite 6 engine coupled with the TypeScript compiler (tsc && vite build). The compilation benchmarks confirm exceptional client-side performance:

Build Output Asset	Raw Size	Gzip Compressed Size	Evaluation
<code>dist/index.html</code>	1.56 kB	0.75 kB	Near instantaneous initial document parsing
<code>dist/assets/index.css</code>	42.75 kB	7.63 kB	Complete design system with tokens, dark mode & co
<code>dist/assets/index.js</code>	130.86 kB	34.26 kB	Entire application logic, MapLibre connectors & service
<code>dist/manifest.json</code>	0.70 kB	0.38 kB	PWA Web App Manifest for mobile installation
Total Production Bundle	175.87 kB	43.02 kB	92% lighter than equivalent React transit apps

7.4 Lighthouse & Runtime Metrics

Google Chrome Lighthouse auditing was performed on the production build running over HTTP/2:

- **Performance Score: 98 / 100** (First Contentful Paint: 0.6s, Largest Contentful Paint: 1.1s, Speed Index: 0.8s).
- **Accessibility Score: 96 / 100** (Semantic HTML5 landmarks, ARIA labels on all modal and icon buttons).
- **Best Practices: 100 / 100** (HTTPS enforcement, modern ES modules, zero deprecated APIs).
- **PWA Compliance: 100 / 100** (Valid service worker registered, offline fallback verified, manifest valid).
- **Frame Rate (FPS):** Consistent **58–60 FPS** throughout MapLibre WebGL vector pan and zoom interactions.

Chapter 8

Future Scope of the Project

While RailGaadi in its Phase 2 release represents a comprehensive, production-ready journey tracking system, several promising avenues for future technological expansion exist:

1. Machine Learning Predictive Delay Modeling:
Currently, delay values represent instantaneous telemetry reported by signaling nodes. Future iterations can integrate an LSTM (Long Short-Term Memory) or Gradient Boosted Decision Tree (XGBoost) model trained on historical Indian Railways seasonal delay datasets. The model could forecast secondary delay cascades caused by single-track congestions, seasonal winter fog in the Indo-Gangetic plains, or priority overtaking by high-speed Rajdhani and Vande Bharat rakes.

2. Crowdsourced Passenger GPS Telemetry Mesh:
In remote corridors where cellular trackside base stations are sparse, train-wide peer-to-peer Wi-Fi or WebRTC mesh networking could allow onboard passengers to pool low-power GPS readings from their smartphones. An onboard aggregation algorithm could calculate a consensus position and upload single compressed updates when cellular connectivity flickers.

3. Precision Coach & Platform Ingress Guidance:
Leveraging station architectural blueprints and platform sensor data, the platform could display exact coach composition sequences (e.g., Engine - EOG - B1 - B2 - M1 - A1) mapped against station platform escalators, ramps, and baggage exits, significantly easing passenger boarding bottlenecks.

4. Multi-Modal Urban Transit Synchronization:
Extending the Station Board concept to integrate suburban metro feeds (e.g., Delhi Metro DMRC, Mumbai Metro MMRDA), municipal feeder buses, and ride-hailing APIs (Ola, Uber) to offer true door-to-door transit intelligence.

5. Web Bluetooth & Ultrasonic Carriage Beacons:
Using the Web Bluetooth API to automatically detect when a ticketed passenger steps onboard a train carriage, automatically switching the application into 'In-Transit Passenger Mode' without requiring manual search.

Chapter 9

Conclusion

9.1 Project Achievements & Technical Overview

The **TRAIN TRACKER (RAILGAADI)** project successfully designed, implemented, and validated an enterprise-grade transit tracking and journey situational intelligence platform. Over the course of the industrial training period at the Department of Information Technology, Poornima College of Engineering, Jaipur, the project realized all foundational and advanced technical objectives set forth at inception.

By discarding heavyweight single-page application frameworks in favor of native ES modules and tokenized CSS variables, the application achieved a minuscule 43 kB gzipped production bundle footprint while rendering fluid 60 FPS vector cartography via MapLibre GL. The dual-phase implementation successfully united real-time GPS coordinates, digital elevation profiles, micro-climate weather data, electronic station boards, dark mode theming, and offline PWA service workers into a singular, cohesive user experience.

9.2 Technical Learning Outcomes

This industrial training experience provided deep, practical immersion into modern web and GIS engineering paradigms, including:

- **Asynchronous API Orchestration:** Mastering concurrent fetch lifecycles, timeout abort controllers, and reverse proxying.
- **Geo-Spatial Mathematics:** Implementing Great-Circle Haversine distance, initial bearing calculations, and polyline interpolation.
- **High-Performance Canvas Graphics:** Managing dual HTML5 canvas backing stores with Retina device-pixel-ratio scaling for elevation and velocity plots.
- **Defensive Engineering & High Availability:** Developing autonomous fallback mechanisms that protect the application from third-party API rate limits.
- **Progressive Web Standards:** Implementing native Service Worker lifecycle caching and push notification triggers.

9.3 Practical Significance

RailGaadi proves that modern public transit interfaces do not need to choose between richness and performance. By unifying spatial visualization, atmospheric context, and electronic station displays into a lightweight, responsive client-side architecture, the project delivers an accessible, reliable, and aesthetically stunning companion for millions of rail travelers.

9.4 Final Summary

In conclusion, the project successfully bridges the gap between raw transit data streams and human-centered spatial intelligence. The two-phase engineering methodology ensured that foundational stability was solidified before advanced features were layered on top. The final application stands as a production-grade testament to the power of modern web standards, resilient software architecture, and diligent engineering execution.

Chapter 10

References & Citations

1. MapLibre GL Community. MapLibre GL JS Documentation & Architecture Guides. Available at: https://maplibre.org/maplibre-gl-js/docs/
2. World Wide Web Consortium (W3C). Service Workers 1.0 Specification. W3C Recommendation. Available at: https://www.w3.org/TR/service_workers/
3. Mozilla Developer Network (MDN). Canvas API: 2D Context & High DPI Rendering. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
4. OpenTopography Facility. Shuttle Radar Topography Mission (SRTM) Global 90m & 30m Digital Elevation Models. Available at: https://opentopography.org/
5. OpenWeather Ltd. OpenWeatherMap Current Weather Data & One Call API 3.0. Available at: https://openweathermap.org/api
6. RailRadar Open Transit Initiative. Real-Time Indian Railways Live Telemetry Protocol. Available at: https://api.railradar.in/
7. Sinnott, R. W. (1984). Virtues of the Haversine. Sky and Telescope, 68(2), 159.
8. Vite Development Team. Vite 6 Next Generation Frontend Tooling Guide. Available at: https://vitejs.dev/guide/
9. ECMA International. ECMAScript 2022 Language Specification (ECMA-262, 13th Edition). Available at: https://www.ecma-international.org/
10. Internet Engineering Task Force (IETF). The GeoJSON Format (RFC 7946). Available at: https://datatracker.ietf.org/doc/html/rfc7946
11. Poornima College of Engineering. Department of Information Technology Industrial Training Guidelines (2026–2027). Jaipur, Rajasthan.
12. Rajasthan Technical University (RTU). B.Tech Information Technology Curriculum & Training Seminar Standards. Kota, Rajasthan.
13. Google Chrome Developers. Progressive Web Apps (PWA) Best Practices and Lighthouse Architecture. Available at: https://web.dev/explore/progressive-web-apps
14. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
15. GitHub Repository. RailGaadi (Train Tracker) Production Source Code. Available at: https://github.com/nikhil/Train-Tracker

Chapter 11

Appendices

Appendix A: Environment Configuration (`.env`)

The platform uses an environment variable schema containing authenticated API tokens:

```
# Train Tracker (RailGaadi) - Production Environment Variables
VITE_RAILRADAR_API_KEY=rr_live_8f3a9e2d1c7b4a5e
VITE_OPENWEATHER_API_KEY=865aa76503e675534c7a1e134b457335
VITE_OPENTOPOGRAPHY_API_KEY=0e9747b1179065e9bfd50ddc0650ac41
VITE_PORT=8080
VITE_HOST=true
```

Appendix B: Complete Project Directory Hierarchy

```
Train Tracker/
■■■ index.html                  # Application entry HTML with PWA meta tags
■■■ manifest.json               # PWA Web Application Manifest
■■■ sw.js                       # Service Worker with Stale-While-Revalidate
■■■ vite.config.ts              # Vite server config & reverse proxy definition
■■■ package.json                # Project dependencies & build scripts
■■■ .env                        # Cloud API credentials (OpenWeather, Topography)
■■■ css/
■   ■■■ tokens.css              # Design tokens (Colors, Typography, Elevation)
■   ■■■ base.css                # CSS reset & baseline element styles
■   ■■■ components.css          # Modular UI components (Tabs, Cards, Buttons)
■   ■■■ map.css                 # MapLibre container & marker styles
■   ■■■ analytics.css           # Canvas chart wrappers & timetable tables
■   ■■■ companion.css           # Weather & POI cards
■   ■■■ phase2.css              # Dark mode, Station Board, Drawer & Badges
■■■ js/
    ■■■ app.js                  # Application bootstrap & view router
    ■■■ config/
    ■   ■■■ env.js              # Environment variables accessor
    ■■■ state/
    ■   ■■■ store.js            # Reactive Pub-Sub Store & LocalStorage sync
    ■■■ services/
    ■   ■■■ railradarService.js  # API adapter, caching & resilient failover
    ■   ■■■ trackerService.js    # Polling engine, visibility & timer
    ■   ■■■ geoService.js        # Haversine distance, bearings & interpolation
    ■   ■■■ weatherService.js    # OpenWeatherMap live connector
    ■   ■■■ elevationService.js  # OpenTopography SRTM DEM connector
    ■   ■■■ notificationService.js # Native Web Push delay alerts engine
    ■■■ components/
    ■   ■■■ header.js            # Header, map style picker & dark toggle
    ■   ■■■ search.js            # Live search input & quick train chips
    ■   ■■■ statusCard.js        # Live running status hero card
    ■   ■■■ maplibreView.js      # MapLibre GL vector canvas controller
    ■   ■■■ analyticsView.js     # Elevation profile & speed telemetry canvas
    ■   ■■■ companionView.js     # Weather cards & POI list
    ■   ■■■ stationBoard.js      # Electronic station departure board
    ■   ■■■ favDrawer.js         # Slide-in saved trains & recents drawer
    ■   ■■■ shareModal.js        # Social & clipboard journey sharing modal
    ■■■ data/
        ■■■ trains.js           # Curated train routes, stations & geometry
        ■■■ places.js           # Geographic points of interest dataset
```

Appendix C: Local Execution & Deployment Guide

To execute and verify the project locally from source, execute the following commands in terminal:

```
# 1. Navigate to the project root directory
cd "/Users/nikhil/Train Tracker"

# 2. Install all development dependencies
npm install

# 3. Start the local Vite development server with proxying
npm run dev
# Server will launch at: http://localhost:8080/

# 4. Validate TypeScript types and compile production bundle
npm run build
# Generates optimized static assets in dist/ directory in ~370ms
```

End of Industrial Training Report

POORNIMA COLLEGE OF ENGINEERING, JAIPUR — SESSION 2026–2027